

System Requirements Specification (SRS)

Real-Time AR Soil/Crop Analysis System

Pixel Hunt Challenge Evolution

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for the Real-Time AR Soil/Crop Analysis System, which evolves the Pixel Hunt Challenge from mouse-based interaction to camera-based AR analysis while maintaining the same data format and structure.

1.2 Scope

System Name: Real-Time AR Soil/Crop Analysis System

Key Transformation:

FROM: Mouse hover on 2D map → Static pixel data display
TO: Camera view of ground → Real-time AR data overlay

Core Capabilities:

- WebXR plane detection using Babylon.js
- AI-based terrain and crop classification
- Real-time GPS-to-pixel mapping
- NASA satellite data integration
- Pixel Hunt data format preservation

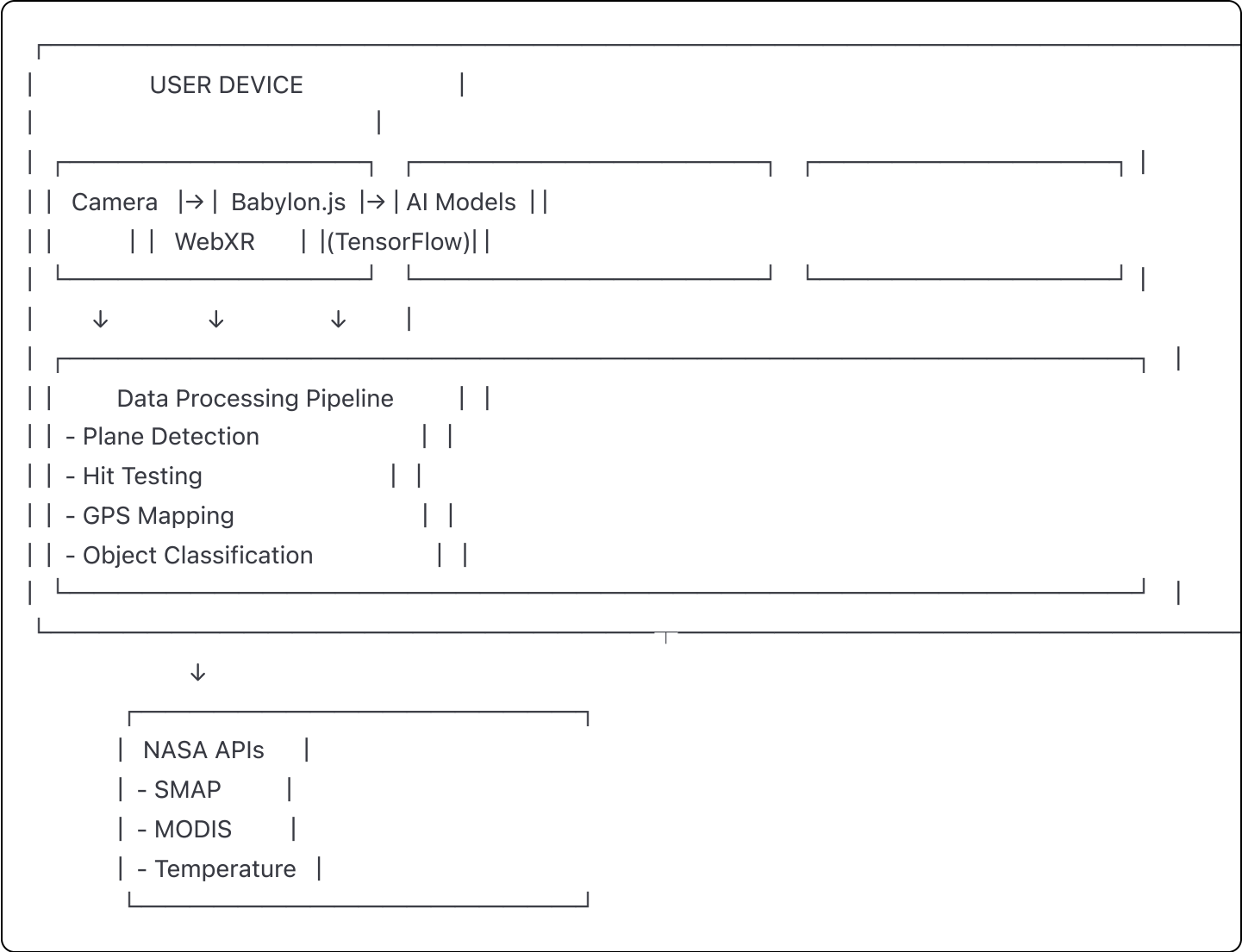
1.3 Definitions and Acronyms

Term	Definition
Pixel Hunt	Original mouse-based system showing pixel information
Plane Detection	WebXR technology to identify flat surfaces
Hit Testing	AR technique to determine 3D point from screen touch
NDVI	Normalized Difference Vegetation Index
Babylon.js	3D/WebXR engine replacing Three.js

Term	Definition
TensorFlow.js	Machine learning library for browser

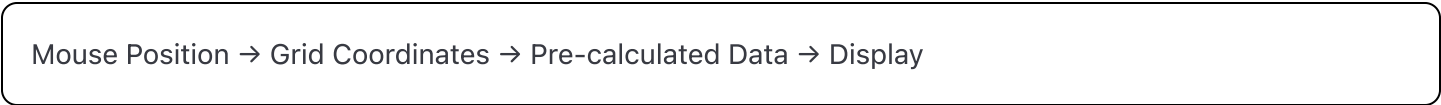
2. System Overview

2.1 System Architecture



2.2 Data Flow Comparison

Original Pixel Hunt:



New AR System:

Camera Frame → Plane Detection → Hit Point → GPS Coordinate →
AI Analysis + NASA Data → Pixel Grid Conversion → Same Display Format

3. Functional Requirements

3.1 Plane Detection Requirements

FR-PD-001: WebXR Initialization

Description: System shall initialize Babylon.js WebXR session for AR **Acceptance Criteria:**

- WebXR session starts within 2 seconds
- Camera feed displays correctly
- AR mode activated successfully

```
javascript  
  
// Implementation Reference  
const xr = await scene.createDefaultXRExperienceAsync({  
  uiOptions: {  
    sessionMode: "immersive-ar",  
    referenceSpaceType: "local-floor"  
  }  
});
```

FR-PD-002: Ground Plane Detection

Description: System shall detect horizontal ground planes **Acceptance Criteria:**

- Detects planes within 1 second of pointing at ground
- Distinguishes ground from walls/ceiling (angle < 30°)
- Minimum detectable area: 1 m²

FR-PD-003: Plane Classification

Description: System shall classify detected planes **Priority:** High **Input:** Plane normal vector

Output: Classification (ground/wall/ceiling) **Algorithm:**

```
angle = acos(dot(planeNormal, upVector))
if angle < 30°: GROUND
else if angle > 150°: CEILING
else: WALL
```

3.2 Object Classification Requirements

FR-OC-001: AI Model Loading

Description: System shall load TensorFlow.js models on startup **Models Required:**

1. Terrain Classifier (soil/grass/crop/concrete/water)
2. Crop Identifier (corn/wheat/soybean/pasture/bare)
3. Health Analyzer (0-100% score)

Performance Criteria:

- Model size: <10MB each
- Load time: <5 seconds total
- Memory usage: <150MB

FR-OC-002: Real-time Frame Analysis

Description: System shall analyze camera frames in real-time **Processing Pipeline:**

```
Camera Frame (30fps) →
Frame Skip (every 3rd) →
Resize (224x224) →
AI Inference (<100ms) →
Result
```

FR-OC-003: Ground vs Sky Detection

Description: System shall distinguish ground from sky **Method:** Color histogram + edge detection
+ AI classification **Accuracy Target:** >95%

3.3 Data Integration Requirements

FR-DI-001: GPS Coordinate Mapping

Description: Convert AR hit point to GPS coordinates **Formula:**

```
javascript
```

```
// GPS Calculation
latOffset = hitPoint.z / 111111 // meters to degrees
lonOffset = hitPoint.x / (111111 * cos(deviceLat))
targetGPS = {
  lat: deviceLat + latOffset,
  lon: deviceLon + lonOffset
}
```

FR-DI-002: NASA Data Retrieval

Description: Fetch satellite data for GPS location **Data Points:**

- Soil moisture (SMAP)
- NDVI (MODIS)
- Surface temperature **Cache Strategy:** 5-minute TTL per coordinate

FR-DI-003: Pixel Hunt Format Compatibility

Description: Maintain exact data structure from original system

```
javascript
```

```
// Required Output Format
{
  pixel: [x, y], // Grid position
  moisture: 24.8, // Percentage
  ndvi: 0.433, // Index value
  temperature: 21.3, // Celsius
  type: "pasture", // Classification
  health: 80, // Percentage
  index: 81 // Grid index
}
```

3.4 Hit Testing Requirements

FR-HT-001: Continuous Hit Testing

Description: System shall perform hit testing at camera center **Update Rate:** 10Hz minimum **Hit Types:** Plane intersections only

FR-HT-002: Hit Point Visualization

Description: Display reticle at hit point **Visual Specs:**

- Yellow torus indicator
- 15cm diameter
- Follows ground contours

FR-HT-003: Hit Validation

Description: Validate hit points are on ground **Validation Criteria:**

- Normal vector within 36° of vertical
- Distance < 10 meters from device
- On detected plane

3.5 Display Requirements

FR-UI-001: AR Overlay Display

Description: Show data panel at hit point location **Display Format:**

| Pixel [9, 6] |

| Moisture: 24.8% |

| NDVI: 0.433 |

| Temp: 21.3°C |

| Type: pasture |

| Health: 80% |

| Index: 81 |

FR-UI-002: Real-time Updates

Description: Update display as camera moves **Update Rate:** 10Hz **Smoothing:** Linear interpolation between updates

FR-UI-003: Visual Consistency

Description: Match original Pixel Hunt visual style **Requirements:**

- Monospace font
- Green text on black background
- Same data precision (1 decimal for %, 3 for NDVI)

4. Non-Functional Requirements

4.1 Performance Requirements

NFR-PERF-001: Frame Rate

Requirement: Maintain 30 FPS minimum **Target:** 60 FPS on high-end devices **Measurement:** Chrome DevTools Performance Monitor

NFR-PERF-002: Inference Latency

Requirement: AI model inference < 100ms **Breakdown:**

- Frame extraction: 10ms
- Preprocessing: 10ms
- Model inference: 70ms
- Postprocessing: 10ms

NFR-PERF-003: Data Update Latency

Requirement: Total pipeline < 200ms

Camera → AI Analysis → NASA Data → Display

10ms 100ms 80ms 10ms

NFR-PERF-004: Battery Consumption

Requirement: <15% battery per 10 minutes **Optimization Strategies:**

- Frame skipping
- Model quantization
- GPU acceleration

4.2 Reliability Requirements

NFR-REL-001: Plane Detection Success Rate

Requirement: >95% detection on suitable surfaces **Test Conditions:**

- Indoor/outdoor lighting
- Various ground textures

- Different viewing angles

NFR-REL-002: Classification Accuracy

Requirement: >85% correct classification **Test Dataset:** 10,000 labeled ground images

NFR-REL-003: System Stability

Requirement: No crashes in 30 minutes continuous use **Memory Limit:** <512MB RAM usage

4.3 Compatibility Requirements

NFR-COMP-001: Device Support

iOS Requirements:

- iPhone XS or newer
- iOS 14.5+
- Safari 14.5+

Android Requirements:

- ARCore compatible devices
- Android 8.0+
- Chrome 89+

NFR-COMP-002: WebXR Support

Required Features:

- immersive-ar session
- plane-detection
- hit-test
- local-floor reference space

4.4 Usability Requirements

NFR-USE-001: Learning Curve

Requirement: Users understand system in <2 minutes **Test:** First-time user success rate >90%

NFR-USE-002: Error Recovery

Requirement: Graceful degradation when AR unavailable **Fallback Modes:**

- 1. Camera-only mode (no AR)
- 2. Manual coordinate entry
- 3. Static demo mode

4.5 Security Requirements

NFR-SEC-001: Data Privacy

Requirement: No image data stored or transmitted **Implementation:**

- All processing on-device
- Only GPS coordinates sent to NASA API
- No user tracking

NFR-SEC-002: API Security

Requirement: Secure NASA API communication **Methods:**

- HTTPS only
- API key rotation
- Rate limiting

5. System Constraints

5.1 Technical Constraints

Constraint	Description	Impact
WebXR Availability	Limited browser support	Requires fallback
Model Size	<30MB total for all models	Affects accuracy
GPS Accuracy	±5-10 meters typical	Coordinate precision
Network Dependency	NASA API requires internet	Needs caching

5.2 Babylon.js Specific Constraints

javascript

```
// Babylon.js version requirements
{
  "@babylonjs/core": "^6.0.0",
  "@babylonjs/gui": "^6.0.0",
  "@babylonjs/loaders": "^6.0.0",
  "@babylonjs/materials": "^6.0.0"
}
```

5.3 AI Model Constraints

TensorFlow.js Limitations:

- WebGL backend required
- INT8 quantization for mobile
- Single-threaded inference
- 224x224 input size fixed

6. Interface Requirements

6.1 External Interfaces

NASA API Interface

```
http

GET /api/smap/moisture?lat={lat}&lon={lon}

Response: {
  "moisture_percent": 24.8,
  "timestamp": "2025-01-15T12:00:00Z",
  "quality": "good"
}
```

WebXR Interface

```
javascript

navigator.xr.requestSession('immersive-ar', {
  requiredFeatures: ['plane-detection', 'hit-test'],
  optionalFeatures: ['dom-overlay', 'light-estimation']
});
```

6.2 Data Formats

Input: Camera Frame

- Format: RGBA pixel array
- Resolution: Device native
- Frame rate: 30 FPS

Output: Pixel Hunt Data

```
typescript

interface PixelHuntData {
  pixel: [number, number];
  moisture: number;    // 0-100
  ndvi: number;        // -1 to 1
  temperature: number; // Celsius
  type: string;         // Classification
  health: number;       // 0-100
  index: number;        // Grid index
}
```

7. Acceptance Criteria

7.1 Functional Tests

Test	Criteria	Pass Condition
Plane Detection	Time to detect	<1 second
Classification	Accuracy	>85% correct
Data Display	Update rate	≥10Hz
GPS Mapping	Error margin	<10 meters

7.2 Performance Benchmarks

```
javascript
```

```
// Performance test suite
describe('Performance Benchmarks', () => {
  test('Frame rate maintained', async () => {
    const fps = await measureFPS(30000); // 30 seconds
    expect(fps.average).toBeGreaterThan(30);
    expect(fps.minimum).toBeGreaterThan(25);
  });

  test('Inference latency', async () => {
    const latencies = await measureInference(100); // 100 samples
    expect(latencies.p95).toBeLessThan(100); // ms
  });

  test('Memory usage', async () => {
    const memory = await monitorMemory(600000); // 10 minutes
    expect(memory.peak).toBeLessThan(512 * 1024 * 1024); // 512MB
  });
});
```

7.3 User Acceptance

- Users can identify ground successfully: >95%
- Data appears within 2 seconds: Required
- Values match expected ranges: Required
- No crashes in typical use: Required

8. Traceability Matrix

Requirement	Implementation	Test Case	Status
FR-PD-001	BabylonARScene.ts	TC-AR-001	
FR-PD-002	PlaneDetector.ts	TC-AR-002	
FR-OC-001	AIModelManager.ts	TC-AI-001	
FR-DI-001	GPSTMapper.ts	TC-GPS-001	
FR-DI-003	DataFormatter.ts	TC-DATA-001	
NFR-PERF-001	FrameProcessor.ts	TC-PERF-001	

Appendix A: Babylon.js Configuration

```
javascript

// Optimal Babylon.js settings for AR
const engineOptions = {
  preserveDrawingBuffer: true,
  stencil: true,
  antialias: true,
  powerPreference: "high-performance",
  doNotHandleContextLost: true,
  audioEngine: false // Not needed for AR
};

const sceneOptions = {
  useGeometryUniqueldsMap: true,
  useMaterialMeshMap: true,
  useClonedMeshMap: true,
  virtual: false
};
```

Appendix B: Model Specifications

Model	Input Size	Output	Size	Latency
Terrain Classifier	224×224×3	6 classes	8.5MB	35ms
Crop Identifier	224×224×3	6 classes	7.2MB	30ms
Health Analyzer	224×224×3	1 value	6.8MB	25ms

Appendix C: Grid System Mapping

GPS Coordinate → Pixel Grid Conversion:

- Grid cell size: 0.01° (~1.1km at equator)

- X (longitude): floor(lon / 0.01)